NPLify · P0 Technical Baseline

Deliverable 9 — Cutover & Opening-Entries Approach · Draft v5.9

Numbered with the ERD edition it is built against.

New XP Technologies Limited

11 October 2026 · Confidential

Status: NPL's plan of 11 October 2026, written out against ERD & Data Model Draft v5.9, the Ledger Posting Design v5.9, the Configuration Schema v5.9 and the Architecture & Stack Note v5.9. NPL runs on spreadsheets today. The plan is to start using NPLify on a staging instance: after every change to a spreadsheet the team makes the same change in the app, for some days, to see that things go in the right direction; earlier deals are added so the history is in one place; when everything is right the database is copied to production and the production app is used from a quiet Sunday. This document gives that plan its steps, its checks and its arithmetic — above all the opening entries, which are what makes the first live posting land on a correct balance.
Audience: NPL Management and Finance, who run the parallel days and sign the go/no-go, and the New XP engineering team.

---

1 · The approach in one page

Three things have to be true on the Sunday evening: the configuration in the app describes NPL's business as it is (Configuration Schema v5.9), the ledger's balances equal the spreadsheets' balances to the cent, and the team has already run real days in the app and trusts it. The plan gets there without touching production until the last day:

1. **Prepare.** Load the configuration (clients, projects, partners, pairs, fee structures, parties, accounts, templates, users and roles) into the staging instance from the import format; validate it against the schema rules; fix the spreadsheets' inconsistencies that the validation surfaces before going further.
2. **Open the books.** Choose the opening date: the close of the last business day before the parallel run. Take every balance the spreadsheets show at that close and post it as opening entries (Section 4). Reconcile: the app's balances now equal the sheets'.
3. **Run in parallel on staging.** For about two weeks the spreadsheets stay the system of record and the app is run beside them: every change made to a sheet is made in the app the same day, by the same person, through the normal screens and approvals. Finance reconciles every evening (Section 6); every difference goes in the discrepancy log and is explained before the next day starts.
4. **Backfill alongside.** Earlier deals — closed before the opening date — are loaded as imported records so they can be found and reported, without postings (Section 5).
5. **Go / no-go on the Friday.** The exit criteria of Section 7 are met or the run is extended; Management signs.
6. **Sunday.** Freeze the sheets after Friday's close; final reconciliation on Saturday; on Sunday reset production once, copy the staging database and the file bucket to it, verify, point the users at production. Monday is the first live day; the sheets stay read-only for four weeks as the fallback; the staging instance is removed.

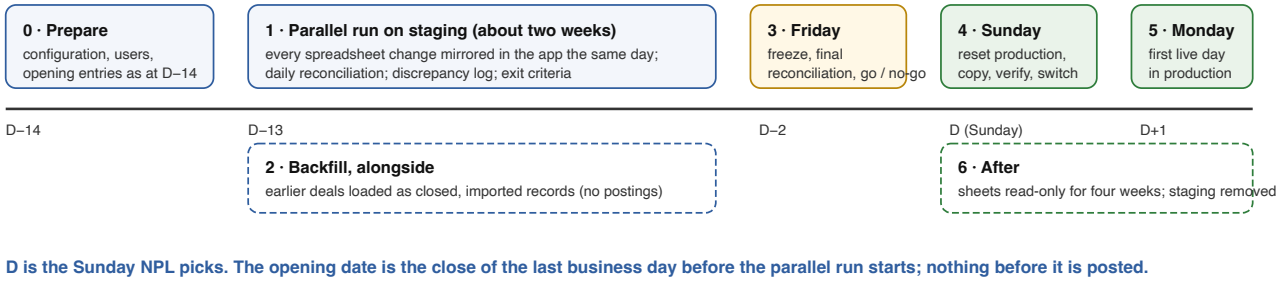


Figure 1 — The cutover in six steps. Only steps 3 and 4 touch production; everything before them is on the staging instance and reversible.

What the parallel run proves, and what it does not. It proves that the configuration prices and routes NPL's real deals the way NPL does, that the postings of each day reproduce the sheets' balances, and that the team can run its day in the app. It does not prove the ledger's arithmetic — that is the Calculation Specification's acceptance vectors and the Ledger Posting Design's worked transactions, which the build already passes — and it does not test every pattern of the Scenario Catalogue: the days will bring whichever deals they bring. Patterns that do not occur in the run are exercised once each on staging as scripted test deals before the go/no-go (Section 7).

2 · Starting point: the spreadsheets

NPL's workbooks are the source of every figure the cutover loads, and the yardstick of every check. Three kinds of content are taken from them:

In the sheets	Becomes in NPLify	Notes
Master data	CLIENT, PROJECT, PARTNER (with cutoffs and holidays), CURRENCY_PAIR and rate chains, FEE_STRUCTURE and FEE_TIER per partner × pair, FEE_OVERRIDE, SENDER with allow-lists, RECEIVER_GROUP, RECEIVER, RECEIVING_ENTITY and accounts, PARTNER_ENTITY and endpoints, DESTINATION_APPROVAL, MARKUP_SHARE and PARTNER_REBATE rules, MESSAGE_TEMPLATE, SYSTEM_SETTING, USER / ROLE / ROLE_RIGHT	The Project Context and Payment methods tabs, the fee table and NPL's answers of 8 October are the sources; the Configuration Schema's import format is the vehicle; its Section 6 rules are the validation.
Balances at the opening date	Opening entries (Section 4): client money at each partner (converted and not yet converted), owed to receivers by group, sender credits, shortfalls, NPL's pool at each partner, own-wallet balances, rebates and shares receivable or payable, open loss receivables, and for NPL-GR the open vendor and resale invoices	Taken from each sheet's balance lines at the close of the opening date; one transaction per project × currency.
Deals	Open deals at the opening date: created in the app in their current state with their stamped figures (Section 4.3). Closed deals before the opening date: imported records (Section 5). Deals from the opening date on: entered day by day during the parallel run.	The sheet's deal reference becomes DEAL_GROUP.client_ref, so a row in the sheet and a deal in the app are the same thing by name.

The validation will surface things the sheets never had to state: a receiver with no approved destination at a partner, a sender with no allow-list, a fee structure whose parts do not add up, a partner with a bank cutoff but no time zone, a pair with no rate source. Each is resolved with NPL before the run — the run must start from a configuration that passes.

3 · The staging instance and the rules of the parallel run

Staging is a second copy of the production stack in NPL's AWS account (Architecture & Stack Note §8): the same image, its own database and bucket, its own sign-in. It exists for the weeks of the run and is removed after go-live. Four rules make the run meaningful:

1. **The sheets lead, the app follows — the same day.** Whoever changes a sheet makes the same change in the app before the end of their shift, as the person they are (Operations enters, Finance approves, Management escalates), through the normal screens. No bulk entry, no "Finance types everything": the point is that the roles and approvals are exercised.
2. **Real values, real timestamps.** The partner's rate as quoted, the market rate at the time (the app captures its own; the sheet's rate is a check), the real cutoff, the real evidence attached. Where the app's clock would have expired a quote the sheet kept alive, the discrepancy log says so: that is a finding about policy, not an error.
3. **Outside services point at copies.** Rates, hash checks and the authenticator app work as in production. The client journal writes to a *copy* of each client's sheet, never to the client's real sheet, until go-live. No message rendered on staging is sent to a sender or receiver; quote packages are rendered and filed only.
4. **Finance reconciles every evening** with the checks of Section 6, before the next day's first deal. A difference is either fixed in the app (an entry missed or mis-entered), fixed in the sheet (the sheet was wrong — these are findings NPL wants), or explained as a known difference of method (rounding at the pair's unit, a fee part the sheet computes differently) and recorded. A difference that cannot be explained stops the run until it is.

Two things the staging days are not. They are not training on test data — that happens on the local environment before the run starts, with the Ledger Posting Design's worked transactions as the exercises. And they are not optional for Finance: the opening entries and the daily reconciliation are Finance's work, and the go/no-go is signed on Finance's log.

4 · Opening entries

The ledger is append-only and every balance is a sum over postings (invariants 2 and 10), so the balances the sheets show at the opening date must themselves be postings. They are posted as **opening entries**: one `LEDGER_TRANSACTION` per project × currency, on the opening date, with `source_type` Opening and the Finance user who prepared it as `posted_by`, approved by Management before posting like any adjustment. Opening entries obey the same rules as every other transaction — balanced per currency and per ownership tag (invariant 1), a `receiver_group_id` on every PAYABLE line (invariant 7) — which is what makes the first live posting land on a correct balance.

4.1 What is opened, account by account

Account (OWNER.HOLDER.CCY.PURPOSE)	Opening balance taken from the sheet
`CL.<PARTNER>.<CCY>.DUE`	Converted client money resting at the partner, waiting to be settled — the sheet's partner balance per currency. One debit per partner × currency.
`CL.<PARTNER>.<CCY>.COLLECTED`	Verified collections not yet converted, in the collection currency, per open deal.
`CL.OWN.<CCY>.HELD`	Client money in NPL's own wallet at the opening date (an approved reroute or a bridge hop in flight). Expected zero (invariant 3); if not zero, the reroute is created as an open record too.
`CL.<RECEIVER>.<CCY>.SHORTFALL`	Delivered short and still to be made good.
`CL.PROJECT.<CCY>.PAYABLE`	What is owed to the project's receivers, per currency, one credit line per receiver group (the sheet's outstanding payouts by receiver, summed by group). Σ PAYABLE + CREDIT = Σ DUE + COLLECTED + HELD + SHORTFALL + MAKEGOOD per currency, by construction of the model; where the sheet does not balance, the difference is found before posting (4.4).
`CL.<SENDER>.<CCY>.CREDIT`	Overpayments owed back to senders (SENDER_CREDIT rows are created with them).
`CL.PROJECT.<CCY>.MAKEGOOD` · `CO.NONE.<CCY>.LOSS_PAYABLE`	An unfunded make-good from a loss before the opening date, if any (LOSS_EVENT created as approved with its funding fields).
`CO.<PARTNER>.<CCY>.POOL`	NPL's own money at the partner: margin captured and not yet settled, plus anything NPL placed there. EARNINGS_RECEIVABLE rows are created for the margin part so an offset or a settlement can name them later.
`CO.OWN.<CCY>.WALLET`	NPL's own money in its wallets, agreed to the on-chain balance.
`CO.<PARTNER>.<CCY>.REBATE_RECEIVABLE` · `CO.<PARTY>.<CCY>.SHARE_PAYABLE` / `SHARE_RECEIVABLE`	Accrued and not yet settled at the opening date, from the month's working. The accrual rows are created in state accrued so the month-end statement includes them.
`CO.<PARTNER|SENDER>.<CCY>.LOSS_RECEIVABLE`	Agreed loss shares not yet received (LOSS_RECOVERY rows in state agreed).
`CO.<VENDOR>.EUR.VENDOR_PAYABLE` · `CO.<CUSTOMER>.EUR.RESALE_RECEIVABLE` · `CO.PROJECT.EUR.REMIT_CLAIM`	NPL-GR only: the undischarged part of each open vendor and resale invoice, in the invoice currency, with INVOICE rows created so INVOICE_BALANCE and the instalment lines work from day one. The revenue and cost of those invoices were recognised in the sheets' period and are *not* re-posted (4.4).
`CO.NONE.<CCY>.OPENING`	The counter-account of every company line: NPL's net position per currency at the opening date. Proposed as a new purpose (4.4). Never posted to again.

Income and expense accounts (EARN_GROSS, EARN_REBATE, EXP_*, VAR_*, FX_RESULT, RESELL_REVENUE, RESELL_COST) open at zero: the app's profit and loss starts on the opening date; the period before it is the sheets'.

4.2 Worked example — Client 1 (the FRS project) at the opening date

The project of the Configuration Schema's worked example, with illustrative balances. Two transactions, one per currency. The client lines balance among themselves; the company lines balance against the opening-equity account. The build checks both before rendering.

Opening EUR — one transaction, source Opening, posted on the opening date

Account	Side	Amount	Owner	Group	Why
CL.ALI.EUR.DUE	Dr	12,400.00	client	DEFAULT	converted at Ali, waiting to be settled (PROJECT_BALANCE)
CL.ALI.EUR.DUE	Dr	1,850.00	client	DEFAULT	a second deal, converted, awaiting the partner's release
CL.PROJECT.EUR.PAYABLE	Cr	14,250.00	client	DEFAULT	owed to the default group's receivers (GROUP_ENTITLEMENT)
CO.ALI.EUR.POOL	Dr	310.55	company		NPL's margin captured at conversion and still at Ali (EARNINGS_RECEIVABLE)
CO.NONE.EUR.OPENING	Cr	310.55	company		opening equity, EUR (proposed purpose)

Opening USDT — one transaction, source Opening, posted on the opening date

Account	Side	Amount	Owner	Group	Why
CL.ALI.USDT.COLLECTED	Dr	3,000.00	client	DEFAULT	verified, not yet converted (deal D-261010-003 in Collected)
CL.PROJECT.USDT.PAYABLE	Cr	3,000.00	client	DEFAULT	the same deal's obligation, in the collected currency until conversion
CL.SND_A.USDT.CREDIT	Cr	25.00	client	DEFAULT	an overpayment owed back to Sender A (SENDER_CREDIT)
CL.ALI.USDT.COLLECTED	Dr	25.00	client	DEFAULT	the overpaid part, still at Ali
CO.JETON.USDT.REBATE_RECEIVABLE	Dr	48.00	company		September rebate accrued, not yet received from Jeton
CO.OWN.USDT.WALLET	Dr	1,250.00	company		NPL's own money in the Aquanow wallet
CO.NONE.USDT.OPENING	Cr	1,298.00	company		opening equity, USDT (proposed purpose)

Reading the EUR transaction: the two DUE lines are two converted deals at Ali; the PAYABLE line is what their receivers are owed, tagged with the default group, so GROUP_ENTITLEMENT(DEFAULT, EUR) = 14,250.00 and PROJECT_BALANCE(Client 1, Ali, EUR) = 14,250.00 from the first minute. NPL's 310.55 at Ali is the pool, and an EARNINGS_RECEIVABLE row of 310.55 is created with it. In USDT, the 3,000 collected and not yet converted is PAYABLE in USDT until the conversion posts; Sender A's 25 overpayment is a CREDIT with its SENDER_CREDIT row; the rebate Jeton owes and NPL's wallet balance are company assets against USDT opening equity.

4.3 Open deals at the opening date

A deal that is in flight at the opening date is created in the app in its current state, with the fields the state requires stamped as they were: the market rate and partner rate version it was priced on (entered as rows dated when they were captured), the sender rate, fee parts and tier, the partner entity and endpoint, the collection parts verified so far with their evidence, the quote package reference. The opening entries carry its money (COLLECTED or DUE and PAYABLE); the deal's own later events — conversion, settlement, confirmation — then post normally. An open deal is therefore the one place where the opening entry and a deal record must agree: the reconciliation of 4.4 checks that Σ COLLECTED + DUE of open deals in the opening entries equals the amounts on the open deal records.

4.4 Rules and two proposed additions to the ERD

1. **Balanced or not posted.** Where a sheet's client assets at partners do not equal its payables per currency, the difference is investigated before anything posts: it is a missed payout, an unrecorded fee, NPL's money mixed with client money, or a sheet error. It is never plugged with an opening line; the sheet is corrected and the opening entry re-prepared.
2. **Opening equity, per currency.** The chart of accounts has no equity account because the model's company lines always balance against an income, expense or asset line of the same event. An opening balance has no event, so a counter-account is needed: `CO.NONE.<CCY>.OPENING`, kind *equity*, never posted to after the opening transaction. Its reporting-currency value is NPL's net position at the opening date. **Proposed ERD addition:** purpose OPENING in LEDGER_ACCOUNT.purpose and kind equity in LEDGER_ACCOUNT.kind.

3. **Source Opening.** One transaction per project × currency names no deal; it needs its own source. **Proposed ERD addition:** `LEDGER_TRANSACTION.source_type` Opening, pointing at the PROJECT, allowed only on the opening date, prepared by Finance and approved by Management (an approval action `post_opening_entries` in ROLE_RIGHT). Invariant 2 stands: an opening entry is corrected by a reversing transaction like any other.
4. **Invoices are posted once, and the opening entry is that once.** Invariant 44 says NPL-GR's vendor and resale invoices are posted once each; for an invoice open at the opening date, the opening entry carries its undischarged payable or receivable and the sheets' period carries its revenue and cost. The INVOICE row is marked imported so the Invoice source is not posted for it again.
5. **Nothing before the opening date is posted.** The sheets are the books up to the opening date; the app's books begin with the opening entries. The period's margin, variances and fees are read in the sheets, and the app's profit and loss begins at zero.

5 · Backfill: earlier deals

NPL wants the earlier deals in the app so the history is in one place. They are loaded as **imported records**: a DEAL_GROUP and its DEAL legs in state closed (confirmed, returned or voided as the sheet says), with the sheet's reference as `client_ref`, the parties, partner, dates, amounts in and out, the sender rate and the partner rate as the sheet has them, and an `imported` marker with the sheet and row they came from. They carry no postings, no approvals and no evidence beyond what the sheet linked, and the screens show them as imported: searchable, reportable, countable in PROJECT_MONTH_VOLUME for the current month's tier if the opening date falls inside a month (D21), but outside the ledger. The alternative — reconstructing every posting for every past deal — would need rate versions, conversions and settlements the sheets never recorded, and would move nothing in any balance, since the opening entries already carry the result of that history.

- **How far back:** NPL decides; the load is the same per deal, so the question is only how many rows the sheets can supply cleanly. Suggested: the current calendar year, or from each project's start if later.
- **Validation on load:** every imported deal names an existing sender, receiver, partner configuration and pair; amounts are numeric; dates are in order; a deal that fails is listed, not loaded.
- **When:** alongside the parallel run, by Operations, in batches per project; checked by Finance against the sheet's deal count and total per month.
- **Not imported:** past rate snapshots (MARKET_RATE starts with the run), past approvals, past audit. The sheets remain the record for those.

Proposed ERD note. DEAL_GROUP and DEAL gain `imported_from` (sheet and row) — nullable, set only on backfilled rows — and the walkthroughs state that an imported deal has no postings. A small addition, recorded here for the next ERD edition with the two of Section 4.4.

6 · Reconciliation checks

Run by Finance every evening of the parallel run, and once more on the Saturday before go-live against the frozen sheets. Each check names what the app shows, what the sheet shows and the tolerance; a check outside tolerance is a discrepancy-log entry with an owner and a resolution before the next day.

Check	In the app	In the sheets	Tolerance	Owner
Balance at each partner, per currency	PROJECT_BALANCE per project × partner × currency	The sheet's partner balance for the same project and currency	Exact	Finance
Money collected and not yet converted	Σ CL.<PARTNER>.<CCY>.COLLECTED	Open collections on the sheet	Exact	Finance
Owed to receivers, per group	GROUP_ENTITLEMENT	The sheet's outstanding payouts by receiver	Exact	Finance
Open deals and their states	DEAL_GROUP / DEAL lists by state	The sheet's deal register	Same set of references; states agree	Operations
Today's deals	Every deal group created today	Every row added to the sheet today	One-to-one, same amounts, same partner	Operations

Check	In the app	In the sheets	Tolerance	Owner
Sender credits and shortfalls	CL.<SENDER>.CREDIT, CL.<RECEIVER>.SHORTFALL	The sheet's notes column	Exact	Finance
NPL's money at partners and in own wallets	CO.<PARTNER>.POOL, CO.OWN.WALLET, CUSTODY	The sheet's pool and wallet lines; the wallet's on-chain balance	Exact; on-chain within pending transfers	Finance
Margin of the day	EARN_GROSS postings of the day, per deal	The sheet's margin column (or the Calculation Specification's vectors where the sheet has none)	Within the pair's rounding	Finance
Rebates and shares accrued	PARTNER_REBATE_ACCRUAL, MARKUP_SHARE_ACCRUAL	The sheet's monthly rebate and share working	Exact at month end	Finance
Invoices (NPL-GR)	INVOICE_BALANCE	The reseller workbook's open invoices and instalments	Exact	Finance
Invariants	The 44 invariants of ERD v5.9 as database checks	—	Zero violations	Engineering
Visibility	A sample of screens and exports per role	Roles & Visibility v5.9 §6 tests	All pass	Engineering with Management

The discrepancy log is a sheet of its own: date, check, project, app figure, sheet figure, cause (app entry · sheet error · method · policy), resolution, who closed it. It is the evidence the go/no-go is signed on, and it is kept after go-live.

7 · Exit criteria and the go / no-go

1. **Five consecutive business days** with every check of Section 6 inside tolerance at the evening reconciliation, and no discrepancy open for more than one day during the run.
2. **Every project NPL runs today** has had at least one deal through the app during the run, or a scripted test deal on staging where the days brought none (the dormant Evo products need none).
3. **The ten flows of the Scenario Catalogue** have each been exercised at least once on staging, by a real deal or a scripted one: F1 Cash by token (collection or settlement); F2 USDT collection with wallet screening; F3 Receiver wallet screened before settlement; F4 SWIFT collection; F5 SWIFT settlement; F6 Returned transfer; F7 One-time local deposit account; F8 Local payouts for small amounts; F9 Bank cutoff; F10 Internal transfer inside the partner's platform.
4. **Every role has done its own work** in the app: Operations has quoted, verified, instructed and prepared; Finance has approved, confirmed rate versions, reconciled and recorded a recovery or an offset; Management has approved an escalation, a configuration change and a user change.
5. **The visibility tests** of the Roles & Visibility Matrix §6 pass on staging with NPL's real configuration.
6. **The backfill** is loaded and its counts and totals per project and month agree with the sheets.
7. **A restore has been rehearsed:** the staging database restored from a snapshot into a scratch instance, and the invariants checked there.
8. **Management signs** the go/no-go on the Friday, on Finance's discrepancy log and the engineering checklist. A no-go extends the run by whole weeks; nothing is rushed into the Sunday.

8 · The Sunday

When	Who	What
Friday close	Operations, Finance	Last entries of the week mirrored; the sheets are frozen (read-only copy made); the app's day is closed with the evening reconciliation.
Saturday	Finance	Final reconciliation of every check against the frozen sheets; the discrepancy log closed; the opening-entry totals re-checked against the first day's reconciliation (the chain from opening to today still balances).
Sunday morning	Engineering	Production reset once (the single reset of NPL's architecture page): database dropped and recreated empty, bucket emptied of test runs. Staging put in read-only mode. Database snapshot and bucket copied to production; migrations confirmed at the same schema version; the 44 invariants run as checks on production; row counts and balance totals compared with staging — identical, or the copy is repeated.
Sunday afternoon	Finance, Management	Sign in to production: one screen per role compared with staging; PROJECT_BALANCE per partner, GROUP_ENTITLEMENT, CUSTODY and the exception list match Saturday's reconciliation. Client journal switched from the copies to the clients' real sheets. Management records the go-live in the audit log.
Monday	Everyone	First live day in production. The sheets are not written to; whoever reaches for one records the item in the app instead. Finance reconciles Monday evening against Friday's frozen sheets plus the day's deals, as during the run.
After	Engineering	Staging removed after two clean weeks in production (its last snapshot kept 30 days). The frozen sheets stay read-only for four weeks as the fallback, then are archived.

9 · Rollback

Until the Sunday, rollback is nothing: staging is discarded or the run extended. On the Sunday, if the copy or the verification fails, production is left empty, staging goes back to read-write and the run continues; the Sunday is tried again a week later. After Monday, if the app cannot be used for a day, the team works from the frozen sheets plus Monday's app entries (the Architecture note's fallback) and enters the day into the app afterwards with the real timestamps in the evidence; the ledger is never edited to catch up, it is posted. A restore from backup is the last resort and a Management decision: the ledger is then the ledger as of the restore minute, and the gap is re-entered from the sheets.

10 · Roles during the cutover

Role	During the run	On the Sunday
Operations	Mirrors every operational change the same day; loads the backfill in batches; reports what the app would not let them do (a finding either way).	Nothing to do; first live entries on Monday.
Finance	Prepares the opening entries; approves as Finance in the app; reconciles every evening; owns the discrepancy log.	Saturday's final reconciliation; Sunday's sign-in check.
Management	Approves the opening entries, escalations and configuration in the app as Management; reads the log weekly.	Signs the go/no-go on Friday; records the go-live.
New XP engineering	Runs staging; loads and validates the configuration; runs the invariant checks nightly; fixes what the run finds, as releases to staging only.	The reset, the copy, the verification, the switch; on call Monday.

11 · Assumptions and decisions for NPL

1. **Opening date** = the close of the last business day before the parallel run starts; nothing before it is posted (Section 4.4, rule 5).
2. **Two weeks** is the planned length of the run; the exit criteria, not the calendar, end it.
3. **Earlier deals are imported records**, not reconstructed postings (Section 5). If NPL wants postings for history too, the opening date moves back to the start of that history and every deal from it is entered through the screens — many weeks of work for no change in any balance.
4. **Backfill depth**: the current calendar year, or from each project's start if later. NPL decides.
5. **The one reset of production** is the one on NPL's architecture page; the copy from staging replaces "live data after" with "the reconciled run's data, then live data after". The two documents agree on that basis.
6. **Client journals** write to copies during the run and to the real sheets from the Sunday.
7. **ERD additions** to be made in the next edition: purpose OPENING and kind equity; source Opening with its approval action; `imported_from` on DEAL_GROUP and DEAL; and, from the Architecture note, RATE_SOURCE code MANUAL.
8. **Staging cost**: a second small service and database for the weeks of the run, about the running cost again (Architecture note §8).

Draft v5.9 — for review with NPL. The opening-entries example is balanced by the script that produced this document; its figures are illustrative.